

SSpMV: A Sparsity-aware Framework for Adaptive Sparse Matrix–Vector Multiplication

Guoqing Xiao¹, Member, IEEE, Shijie Zhao, Qin Du¹, Shengle Lin¹, Yuedan Chen¹, Member, IEEE, Wangdong Yang¹, and Kenli Li¹, Senior Member, IEEE

Abstract—Sparse Matrix–Vector Multiplication (SpMV) is a fundamental sparse kernel in both scientific computing and artificial intelligence. However, the performance of existing SpMV algorithms is highly sensitive to diverse sparsity patterns, necessitating an adaptive framework that can effectively extract matrix structures and predict optimal configurations. In this work, we first analyze the performance of parallel SpMV algorithms in various formats. Based on the analysis, we propose Sparsity-aware SpMV (SSpMV), a framework that integrates handcrafted expert features with multimodal representations to automatically determine the optimal algorithm and parameters. At its core is MM-Adapter, a novel multimodal convolutional neural network designed for this selection task. The robustness of SSpMV is validated using a diverse dataset, consisting of nearly the entire SuiteSparse Collection supplemented by finite element matrices. On an Intel Xeon 5120 CPU, SSpMV achieves a selection accuracy of 81.05% and a geometric speedup of $1.26\times$ over the MKL spblas IE. Furthermore, on an NVIDIA A100 GPU, it yields 74.12% accuracy and a $1.31\times$ speedup over the cuSPARSE baseline.

Index Terms—Sparse Matrix, SpMV, Machine Learning.

I. INTRODUCTION

WITH advances in hardware and exponential growth of data, sparse linear algebra has become crucial in engineering, science, data analysis, and artificial intelligence [1]–[4]. A key computational block within sparse computing paradigms [5] is Sparse Matrix–Vector Multiplication (SpMV), defined as $y = Ax$, which is essential for solving sparse linear systems and deep learning applications [6], [7]. The SpMV kernel, frequently utilized in iterative solvers such as Jacobi and Gauss–Seidel methods, requires repeated operations with identical sparse structure matrices [8], [9]. Therefore, optimizing SpMV for multicore architectures is critical due to its significant computational demands.

Achieving high-performance SpMV on multi-core platforms is challenging due to irregular memory access and limited vectorization from uneven nonzero distributions. While various

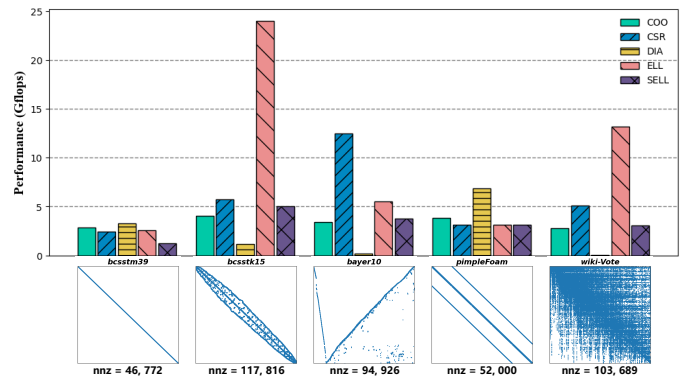


Fig. 1. An observation of sparse matrices from different applications and their SpMV performances in various formats. *bcstsm39*, *bcstsk15*: structural problem, *bayer10*: chemical process simulation, *pimpleFoam*: computational dynamic problem, and *wikiVote*: social network.

storage formats have been proposed to mitigate this [10]–[15], no single format suits all matrices. Figure 1 illustrates the performance variability of SpMV from different applications when running the iterative solver (PETSc), showing up to a $20.8\times$ gap in performance. Thus, it is critical to develop mechanisms that match sparse formats to specific patterns, ensuring optimal SpMV performance across diverse hardware platforms.

Several traditional empirical analytical models [16]–[19] often lack generalization when dealing with large numbers of input matrices. Consequently, machine learning approaches such as decision trees (DT) [20], [21], support vector machines (SVM) [22], multilayer perceptrons (MLP) [23], and convolutional neural networks (CNN) [24], [25] have been adopted for adaptive format selection. Specifically, these methods fail to capture fine-grained structural cues such as local non-zero clustering, tile-wise distribution patterns, and row-level vectorizability that are critical for SpMV performance. However, despite utilizing explicit matrix features, **the implicit information of sparse patterns remains underexplored**. Furthermore, many studies rely on limited or synthetic datasets, yielding unconvincing results for practical large-scale systems.

In this context, we propose SSpMV, a Sparsity-aware SpMV framework designed to address these challenges on both CPU and GPU platforms. SSpMV consists of two key components: 1) a collection of optimized SpMV algorithms for multi-core CPUs, coupled with tuned implementations for GPUs, 2) the intelligent MM-Adapter, which uses a multimodal

Manuscript received xxx, 2026; revised xxx, 2026; accepted xxx, 2026. This work was partially funded by the National Natural Science Foundation of China under Grant 62532005, Grant 62502155, Grant U25A20422, and Grant 62541202. Corresponding author: Shengle Lin.

Guoqing Xiao, Shijie Zhao, Qin Du, Shengle Lin, Wangdong Yang, and Kenli Li are with the School of Computer Science, Hunan University, Changsha 410082, China, and also with the National Supercomputing Center in Changsha, Changsha 410082, China (e-mail: xiaoguoqing@hnu.edu.cn; shijiezhao@hnu.edu.cn; seery313@hnu.edu.cn; lsl036@hnu.edu.cn; yangwangdong@hnu.edu.cn; lkl@hnu.edu.cn).

Yuedan Chen is with the Big Data Institute, Central South University, Changsha 410083, China (e-mail: cheniyuedan@csu.edu.cn).

convolutional model to predict the most efficient algorithm for a given matrix on either platform, surpassing simple format selection. Moreover, in addition to the SuiteSparse Collection [26], we utilize finite element software FreeFEM [27] and computational fluid dynamics software OpenFoam [28] to generate extensive matrices, ensuring that SSpMV is both representative and practical.

Specifically, on the CPU side, SSpMV integrates a variety of general sparse formats, including COO, CSR, DIA [29], CSR5 [13], BCSR [30], ELL and S-ELL (with two variants) [12], [31], and implements several optimization strategies, such as minimizing paddings for vectorization, cache-line alignment, and adapting OpenMP scheduling for load balancing. In contrast, the GPU implementation excludes the inefficient DIA and BCSR formats. Instead, it employs GPU-specific optimizations, including fine-grained load balancing and explicit vectorization using float4 operations [32] to maximize memory throughput and warp efficiency. Additionally, the MM-Adapter incorporates a comprehensive set of expert-designed features and explores sparse structure at different granularities, **constructing a three-modality analysis model that encompasses row vectorization, input locality, and sparse distribution.**

Our experiments on more than 6,000 matrices from various practical sparse systems demonstrate that our SSpMV achieves a geometric speedup of $1.26\times$, with a maximum of $46.8\times$, compared to the Intel MKL Inspector-Executor Sparse BLAS. Meanwhile, on NVIDIA GPUs, SSpMV delivers a geometric speedup of $1.31\times$ over the SpMV function by cuSPARSE, achieving a peak speedup of $3.33\times$. The main contributions of this work are as follows:

- We develop a high-performance Sparsity-aware SpMV library that automatically selects the optimal method from several algorithms, allowing users to integrate new formats and adjust their tuning parameters.
- We design a multimodal convolutional neural network, MM-Adapter, capable of extracting detailed features from each modality of a given matrix, making it suitable for SpMV adaptive tasks.
- We evaluate the performance of nearly all commonly used formats on a large set of real sparse systems, verifying the significant speedup achieved by SSpMV.

II. BACKGROUND AND MOTIVATION

The SpMV operation ($y = Ax$) multiplies a sparse matrix A ($m \times n$) by a dense vector x ($n \times 1$), resulting an output vector y ($n \times 1$). To balance compression efficiency and computational performance, various storage formats have been proposed. This section provides a brief introduction to four fundamental formats: COO, CSR, DIA, and ELL, followed by a discussion of three popular memory-friendly optimized formats: BCSR, CSR5 and S-ELL series.

A. Basic Formats and SpMV Implementations

Figure 2 illustrates a 6×6 sparse matrix represented in four basic formats: COO, CSR, DIA, and ELL, where colored

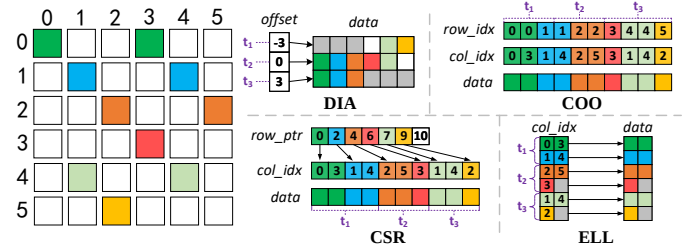


Fig. 2. COO, DIA, CSR, and ELL storage formats of a 6×6 sparse matrix. Their parallel SpMV implementations exhibit different access patterns to data structures by thread t_i .

blocks indicate nonzero elements, white blocks represent zeros, and gray blocks denote paddings. Each format exhibits a distinct multi-threaded access pattern during the parallel computation of SpMV.

COO (coordinate) stores nonzeros as $(row_idx, col_idx, data)$ triples. Its parallel performance is pattern-agnostic since each nonzero is processed independently. Although it leads to random access to the input vector x , this format is particularly effective for large-scale graphical analysis applications [4], [33].

CSR (Compressed Sparse Row) compresses row indices via a row_ptr array to improve memory efficiency, while sharing col_idx and $data$ arrays with COO. However, if nonzeros are unevenly distributed among rows, it can cause load imbalance between threads. Grouping rows by the number of nonzeros (nnz) can mitigate this issue, but irregular access to the input vector persists.

DIA (DIagonal) stores nonzeros along diagonals, using $offset$ and $data$ arrays, making it ideal for diagonally dominant matrices. It offers regular, cache-friendly access patterns per thread. However, if nonzeros are widely scattered outside diagonal positions, significant memory bandwidth is wasted accessing invalid data.

ELL (ELL-PACK) utilizes 2-D col_idx and $data$ arrays, with left-aligned nonzeros to enable dense storage, memory alignment, and coalesced vectorized access. However, when row lengths are unbalanced, it suffers a lot of invalid storage and load imbalance.

B. SpMV with Vectorization and Locality

To efficiently utilize the vector units of modern multi-core processors and optimize memory access patterns in sparse structures, the following optimized formats are also incorporated into SSpMV, as illustrated in Figure 3.

BCSR (Blocked-CSR) [30] divides the matrix into uniform blocks stored in CSR format with dense row-major layout. It exploits nonzero locality and eliminates irregular access to vector x . However, if nonzeros are scattered, BCSR performance degrades due to the increased number of stored zeros.

S-ELL (Sliced-ELLPACK) [12], [31] introduces a chunk width c to ELL, storing rows in cache-line-aligned chunks to reduce padding. The $SELL-c-\sigma$ variant adds a parameter σ to define the size of row segments for pre-reordering, where σ rows in each segment are sorted in descending order based on nnz , while $SELL-c-R$ applies permutations globally. These

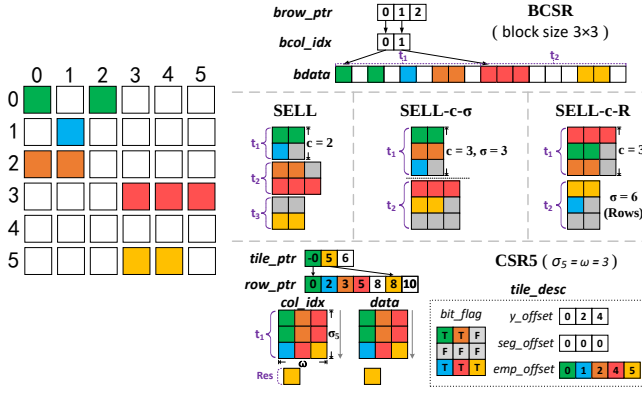


Fig. 3. BCSR, S-ELL, SELL-c- σ , SELL-c-R, and CSR5 storage formats of another 6×6 example sparse matrix.

formats all schedule workloads by chunks, resulting in fewer zero paddings and reduced computations compared to ELL. They efficiently utilize vector units through explicit memory alignment in each dense row chunk.

CSR5 [13] stores *col_idx* and *data* in 2-D tiles for near-optimal load balancing. By aligning σ_5 to cache lines and ω to SIMD length, *tile_desc* is bitwise-compressed into *int32*. This ensures stable throughput across varying sparse patterns.

C. Motivation

As observed in Figure 1, selecting the appropriate storage format is crucial for optimal SpMV performance. Previous work [20], [23]–[25] focuses only on adaptive format selection, ignoring the effect of different parameter configurations on parallel performance. Recent research [21], [22] models only a few vectorized algorithms without covering basic formats. comprehensively exploring all formats and implementations enables exploiting broader performance potential, motivating this work.

To this end, SSPMV incorporates numerous sparse formats and parallel algorithms (Table I). It also implements load-balancing (*Lb*) kernels for basic formats based on per thread workload. Optimized formats like BCSR, CSR5, and SELL-c- σ , use architecture-specific parameters to minimize zero padding, utilize vector units, and enhance memory access. Accordingly, this variety of format options and extensive parameter settings further complicate SpMV performance modeling.

To map a sparse matrix to its optimal algorithm, prior work [20], [21], [30] relies on expert-designed statistics and learning models for auto-selection. These designed features and compressed representations have proven to be insufficient for the complex prediction scenarios of SSPMV, refer to Section 5. To address this, we introduce a multimodal technique analyzing three matrix modalities to capture hidden sparse pattern structures. In summary, our work aims to develop a robust approach for determining the optimal storage format and parameter settings among these well-known implementations.

TABLE I
FORMATS WITH HYPERPARAMETERS USED IN SSPMV

Format	Method	Params	Description
COO	OpenMP, <i>Lb</i>	SCH	SCH is the OpenMP scheduling mode, including Static (St), StaticChunk (StC), Dynamic (Dyn) and Guided (Gd). <i>Lb</i> denotes parallelism with a load-balanced partition by <i>nnz</i> .
CSR	OpenMP, <i>Lb</i>	SCH	
DIA	OpenMP, <i>Lb</i>	SCH	
ELL	OpenMP, <i>Lb</i>	SCH	
S-ELL	S-ELL (<i>Lb</i>)	SCH	<i>c</i> is the row chunk size that affects vector paddings. σ defines the reordering blocks of rows to reduce fillings while trying to keep locality.
	SELL-c- σ	SCH, c, σ	
	SELL-c-R	SCH, c	
BCSR	OpenMP, <i>Lb</i>	SCH, <i>bR, bC</i>	<i>bR, bC</i> control the block size by SIMD width and alignment.
CSR5	AVX-512	σ_5, ω	ω is the size of SIMD unit. σ_5 is defined by cache capacity.

III. ANALYSIS OF PERFORMANCE DISTRIBUTIONS

To characterize the execution of SpMV and demonstrate the potential of different formats and parameters, we conduct extensive benchmarks on the SuiteSparse Collection. Datasets with complex values are excluded, resulting in a total of 2,834 matrices that cover almost all application domains in SuiteSparse. Among these matrices, 2,266 of them contain fewer than 1 million nonzeros, while *nnz* of the remaining matrices ranges from 1 million to 1.9 billion. All implementations listed in Table I are tested for analysis. The performance distributions reveal the following insights:

1. Performance characteristics of the four basic formats.

As illustrated in Figure 4, each SpMV method excels with specific sparse patterns, even among the four basic formats. COO and CSR suit structures with fragmented nonzeros, while ELL outperforms S-ELL when row lengths are similar, as balanced distributions enable good memory efficiency without segmentation or reordering. DIA shows significant speedup ($11.4\times$ to $34.8\times$) for main-diagonalized nonzero distributions. In addition, *Intel MKL dcsr_SpMV* does not perform best on most sparse matrices.

2. The importance of parameter settings.

Figure 5 shows the significant impact of OpenMP scheduling policies (*St*, *StCon*, *Dyn* and *Guided*) on SpMV. Relative to the optimal scheduling strategy, CSR format fluctuates between $0.77\times$ and $1\times$, while the COO experiences up to a $3.1\times$ slowdown. This disparity arises because COO is parallelized by nonzeros, making scheduling more impactful. Given the variability of sparse patterns, no single scheduling policy is universally superior.

3. Explicit load-balanced kernels affect speedup substantially.

As listed in Table I, SSPMV provides various load-balancing implementations based on the *nnz* processed by each thread. Our experimental results indicate that explicit load balancing significantly impacts CSR, ELL, and S-ELL formats. Figure 6 illustrates the speedup achieved by CSR and S-ELL load-balancing optimizations over their original kernels. The algorithms based on row and row-block parallelism are prone to load imbalance due to the skewed distribution of nonzeros. Explicitly balancing the thread workload yields maximum speedup ratios of $2.7\times$ for CSR and $9.8\times$ for S-

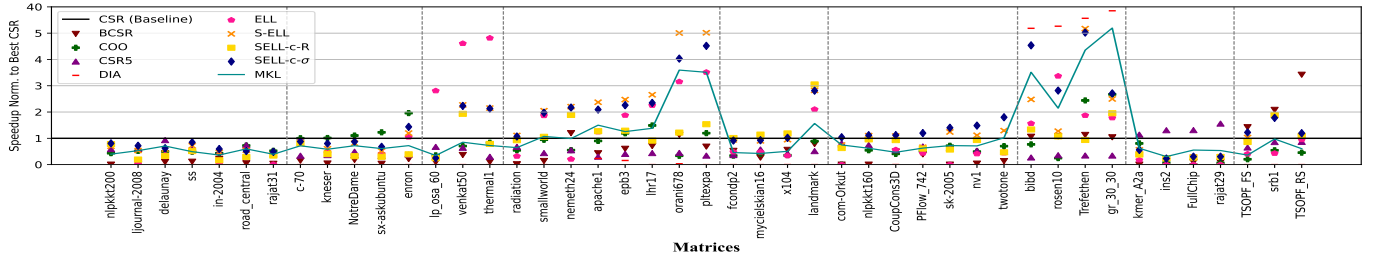


Fig. 4. Speedup of double-precision SpMV in different sparse formats and Intel MKL SpMV kernel over our best implementation of CSR format. The performance results of these SuiteSparse matrices are grouped according to their fastest format.

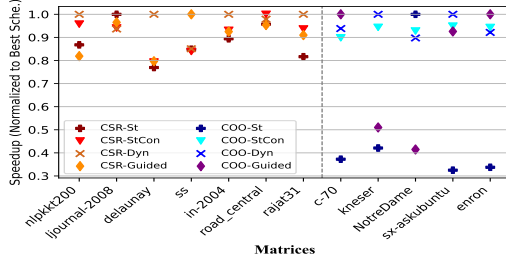


Fig. 5. Performance degradation of different scheduling methods to the best implementation.

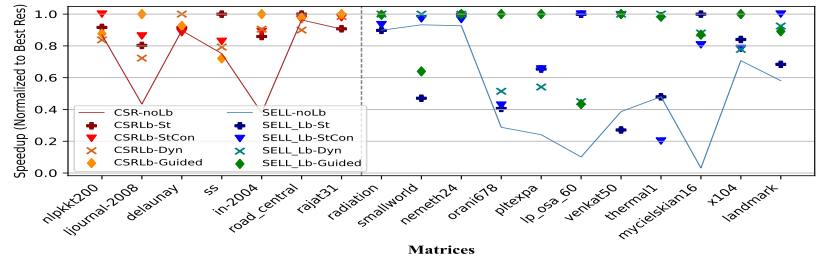


Fig. 6. Speedup of different scheduling strategies with load-balanced optimization over the best original kernel of CSR and S-ELL formats.

ELL. Consequently, we consider introducing explicit load-balanced optimizations in SSpmv.

4. Existing expert-designed metrics are insufficient to characterize sparse patterns. To investigate the effect of parameters and load imbalance on the distribution of the fastest method, we conduct two experiments using analytical features emphasized by previous researchers. These tests are performed on SuiteSparse matrices with nnz ranging from 2^{21} to 2^{29} , encompassing a variety of sparsity patterns.

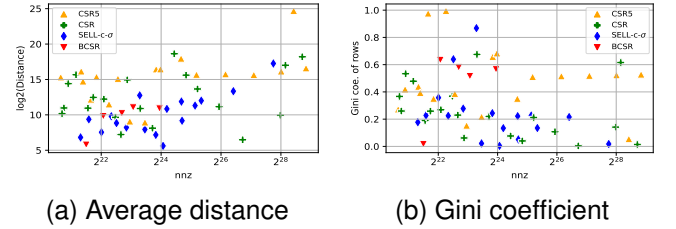


Fig. 8. Distribution of fastest method over row features.

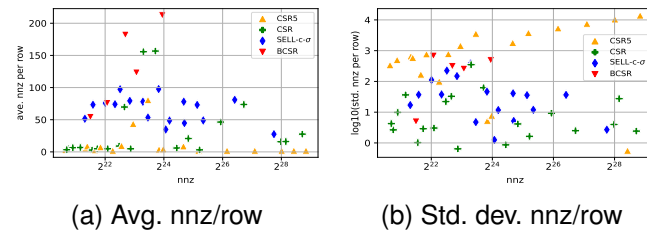


Fig. 7. Distribution of fastest method over row features.

In the first experiment, we consider two widely used features [9], [20], [21], [34] of the nnz distribution per row: the *average number of nnz per row*, and the *standard deviation of nnz between rows*.

From Figure 7(a), we observe that BCSR and SELL-c- σ perform better on datasets with higher nnz per row, while CSR and CSR5 generally prefer sparser patterns. Figure 7(b) reveals a cleaner distribution pattern for the CSR5 format, showing its marked advantage in cases of extreme row nnz imbalance. However, when the workload decreases and nnz becomes similar between rows, it is difficult to predict which method will be the fastest.

In the second experiment, we use *distance* and *Gini coef*-

ficient to describe nonzero locality and skew. *Distance* [23], [25] measures the average gap between consecutive nonzeros in a row, reflecting aggregation degree. The *Gini coefficient* [21], [35] measures the inequality in the distribution of nnz in rows, with values closer to zero indicating a more balanced distribution. Figure 8(a) shows CSR, BCSR, and SELL-c- σ excel when nonzeros are concentrated, while Figure 8(b) indicates CSR5 suits large-scale imbalanced patterns. However, performance becomes unclear when nnz distribution is balanced due to complex interactions among workload, skew, and locality. Overall, predicting the optimal algorithm using only these expert-designed features remains challenging despite format optimizations.

IV. SSPMV OVERVIEW

Since benchmarks demonstrate the potential of diverse SpMV formats, we introduce the Sparsity-aware SpMV Framework (SSpmv) to identify optimal implementations from numerous algorithms (Figure 9). SSpmv integrates expert-designed features with latent nonzero correlations, constructs multimodal feature-label pairs via benchmarking (Table I), and develops the *Multimodal Adapter* (MM-Adapter).

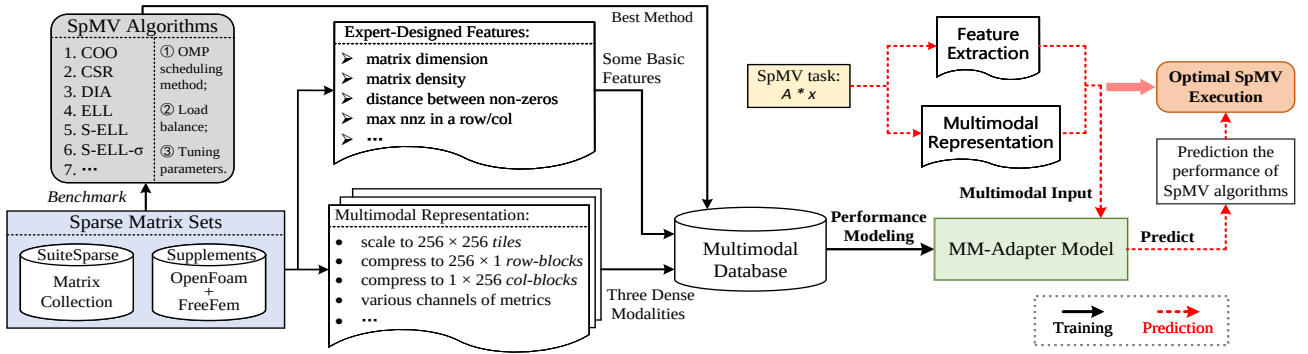


Fig. 9. The SSPMV overview: The black line indicates multimodal features extraction and model training phase, and the red dotted line shows the SSPMV execution process. The MM-Adapter is a multimodal CNN for SpMV performance modeling.

This model fuses expert features and matrix representations to predict the most effective kernel across sparse patterns. To support modeling, we use 6,052 datasets (2,834 SuiteSparse + 3,218 generated 2D/3D matrices), extracting features and benchmarking GFLOPS as ground truth. As shown in Figure 9, SSPMV trains MM-Adapter on this database to predict performance and select the optimal SpMV solution for new tasks.

In conclusion, the SSPMV framework offers a unified interface for CSR format, enabling smart conversion to other formats and algorithms. To enhance extensibility, it allows users to integrate new kernels, thereby improving both performance and robustness. On a specified architecture, SSPMV requires only a one-time offline training of the MM-Adapter, which can then be reused for performance prediction. This approach ensures both convenience and portability.

A. Expert-designed Features

The SSPMV framework relies on an input-aware mechanism that starts with the collection of diverse expert-engineered features. Some quantify macroscopic attributes like matrix dimensions, sparsity, diagonal nnz concentration, and average intra-row nonzero distances, while others capture micro-level characteristics such as memory access patterns, padding overheads, and data skewness. Together, these provide essential data for accurate performance modeling.

Figure 10 illustrates the full expert-designed features utilized in SSPMV. Specifically, we use six features to represent the common sparse structure. Subsequently, to derive finer details, the matrix is segmented at varying granularities. In addition to traditional rows and columns, we divide the sparse matrix into 256×256 tiles, 256×1 row blocks, and 1×256 column blocks, respectively. We define each such segment as a 'bucket' and extract the subsequent six fine-grained features from them:

- 1) nz : Ratio of buckets that are not zero;
- 2) max : Maximum number of nonzeros between buckets;
- 3) μ : Mean of nnz between buckets;
- 4) σ : Standard deviation of nnz between buckets;
- 5) G : The *Gini coefficient* of the nnz among buckets;
- 6) P : The p -ratio of the nnz among buckets. P -ratio means that $p\%$ of the buckets have $(1 - p)\%$ nonzeros.

The first four features are commonly used to characterize the statistical properties of data distributions [21], [23], [30], [36]. The last two features measure equilibrium: for a perfectly balanced situation, the p -ratio is 0.5 and the Gini coefficient is 0. As described in Figure 10, these 40 features provide a preliminary assessment of structures in each granularity, roughly indicative of the memory access patterns of SpMV algorithms.

B. Multimodal Representation

The effective representation of sparse matrices is hindered by two primary obstacles. Firstly, standard CNNs are tailored for fixed-resolution imagery, creating a mismatch with the variable scales of sparse matrices. Second, complex sparsity leads to unique preferences for memory access, scheduling, and architectural factors that existing approaches cannot fully capture [25], [36]. To address these limitations, we propose a novel multimodal representation framework tailored specifically for the heterogeneity of sparse structures.

Unique challenges in SpMV have led us to propose a new interpretation of multimodality in the context of sparse matrix operations. Traditional multimodal learning typically fuses distinct data types [37], [38]. In contrast, our work redefines multimodality within the sparse matrix domain by generating multiple representations from a single input. Specifically, we construct three modalities: **locality**, **distribution**, and **vectorization** to handle the particular requirements of SpMV kernels, corresponding to complex memory access patterns of the input vector, load balancing of nnz and utilization of SIMD instruction, respectively. Figure 11 illustrates mapping a 9×9 matrix into these three modalities. The first modality (1×3 vectors) captures the **locality** of the input vector at column-block granularity. The second modality (3×1 vectors) derived from the row block perspective, emphasizes the **vectorization** characteristics across row chunks. Lastly, the third modality, depicted as 3×3 submatrices, portrays the **distribution** of nonzeros by converting the sparse matrix into dense-tile blocks.

However, this abstraction risks losing fine-grained details. To compensate, we augment each modality with auxiliary channels encoding sparse statistical metrics. As shown in Figure 11, these enhancement channels are:

Granularity	Features	Description
Matrix	row, col, nnz, sparsity	Depict the overall Matrix
	nnz_ratio_in_main_diag	Capture the distribution of nnz on diagonals
	ave_nnz_distance	Capture the locality to the input vector
Rows	nz _R , max _R , μ _R , σ _R , G _R , P _R	Determine the row scheduling method and paddings
Columns	nz _C , max _C , μ _C , σ _C , G _C , P _C	Determine the memory access pattern to input vector
Tiles	nz _T , max _T , μ _T , σ _T , G _T , P _T	Describe the balance of nnz across Tiles
	uniqR, uniqC	Describe the locality inside a Tile
	reuseR, reuseC	Describe the data reuses probability across Tiles
Row Block	nz _{RB} , max _{RB} , μ _{RB} , σ _{RB} , G _{RB} , P _{RB}	Determine cache utilization and vectorization
Col Block	nz _{CB} , max _{CB} , μ _{CB} , σ _{CB} , G _{CB} , P _{CB}	Determine cache utilization and cache-line alignment

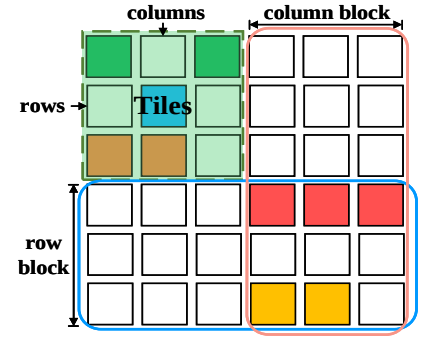


Fig. 10. Expert-designed features and descriptions. The right figure shows the different granularities of feature extraction.

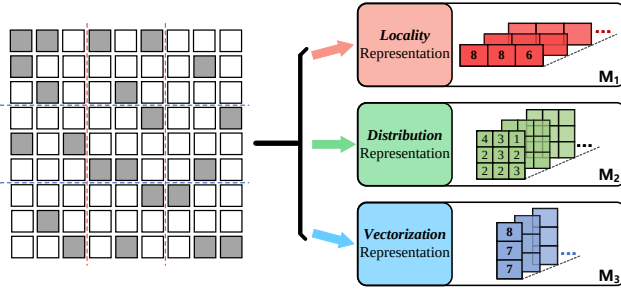


Fig. 11. Three modality representations of a matrix.

- C1: *Nnz* in each segment (M_1, M_2, M_3);
- C2: The maximum *nnz* per row for M_1 and M_2 , and per column for M_3 ;
- C3: The average *nnz* per row for M_1 and M_2 , and per column for M_3 ;
- C4: The standard deviation of *nnz* between rows for M_1 and M_2 , and between columns for M_3 .

Figure 11 shows the details of C_1 . To strike a balance between providing sufficient structural information and limiting the extra overhead of MM-Adapter, we standardize each dimension to a fixed size of 256. During preprocessing, we normalize the feature values of each channel to a standard normal distribution, facilitating more effective learning and accelerating model convergence.

C. MM-Adapter Design

Given the success of CNNs in image domains, we adopt this architecture as the backbone of our *Multimodal Adapter* (MM-Adapter). A key design principle is aligning observation granularities, enabling seamless fusion between hand-crafted statistical features and latent representations learned by the multimodal CNN.

As shown in Figure 12, MM-Adapter takes four inputs from sparse matrix A : three modality views and one expert-engineered feature vector. Specialized convolutional layers process each modality according to its dimensionality: *conv1D* layers handle locality and vectorization modalities to capture long-range dependencies, while *conv2D* layers process the distribution modality for local spatial patterns. Expert-designed features are then integrated with latent features at

matching partitioning granularities. A Feed-Forward Neural Network (FFNN) transforms these features into a dense embedding, which an MLP subsequently maps to the optimal SpMV algorithm.

To construct the matrix labels, we compute the selection probability for each SpMV algorithm as follows:

$$P_i = \frac{G_i}{G_1 + G_2 + \dots + G_N}, \quad (1)$$

where G_i represents the GFlops achieved by the i -th algorithm. Thus, the optimal SpMV implementation is identified as the algorithm with the highest probability, as illustrated in the "Logits" of Figure 12. With this training methodology, the MM-Adapter can be easily adapted to any platform, and new kernels can be integrated into SSpMV to enhance its diversity.

V. EXPERIMENTAL RESULTS

A. Machine

Experiments were conducted on an Intel Xeon Gold 5120 processor (2 sockets, 28 cores total) featuring 32 KB L1, 1 MB private L2, and 19.25 MB shared L3 caches per socket, 256 GB main memory, and AVX-512 support. To validate framework generality, we also utilized an NVIDIA A100-PCIE-40GB GPU comprising 108 Streaming Multiprocessors (SMs), each with up to 164 KB configurable shared memory, and 40 GB high-bandwidth global memory.

B. Dataset

With the advent of exascale computation, sparse linear systems in HPC have grown significantly [39], [40], making SuiteSparse matrices insufficiently representative. Therefore, we retained 2,834 SuiteSparse matrices and generated 3,218 large-scale supplementary matrices using FreeFEM [27] and OpenFoam [28] sample programs (e.g., Laplace3D, Stokes, Potential Flow), totaling 6,052 matrices. Figure 13 shows the properties of the SuiteSparse Collection and our generated datasets. On average, the supplementary matrices are $10.2 \times$ more sparse, have a $16 \times$ larger scale, and exhibit $1.9 \times$ less variation in *nnz* of rows. New datasets effectively enrich large-scale matrices, with sparsity better reflecting the characteristics of modern sparse linear systems. Based on this representative combined dataset, we use 70% for training, 10% for validation, and 20% for testing. For GPU experiments, we use only the SuiteSparse matrices.

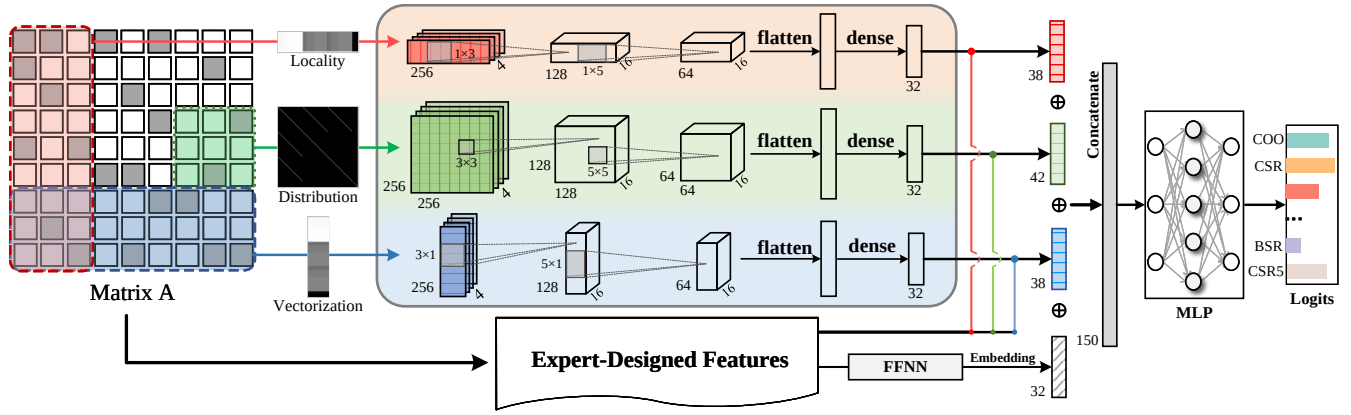


Fig. 12. Details of MM-Adapter design. Multimodal latent features are extracted by CNNs and combined with expert-designed features to model SpMV performance.

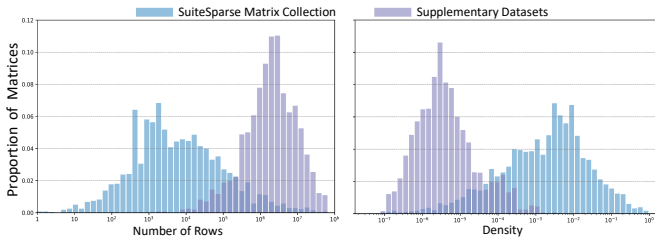


Fig. 13. Property comparisons of two different datasets.

C. Implementations

1) *CPU*: For the various formats and implementations of SpMV source code, we utilize *C++17* and *OpenMP* for shared-memory parallel programming. We use the Intel *icx* compiler with *O3* and *max512f* flags. In addition, we use OpenMP *simd* pragma to implement automatic vectorization in different formats and test four different schedule modes by calling *omp_set_schedule* API. In our experiment, all 28 cores are turned on, and thread affinity is set using KMP_AFFINITY definition, which means that each thread is strictly pinned to a specific physical core. All basic formats use explicit load-balancing optimization strategies.

From Table I, we pick all four scheduling methods for COO and CSR formats, while DIA uses only the St scheduling policy. For the S-ELL (ELL) format family, both StC and Dyn scheduling mechanisms are used, with *c* set to 8 to match the vector width of the machine. The reordering factor σ is chosen from $\{2^9, 2^{14}\}$, aligning with the maximum lengths typically buffered in the L1 and L2 caches, respectively. For BCSR and CSR5, we fix $b_R = \sigma_5 = 16$ and $b_C = \omega = 8$ to match the cache-line width and vector length with only StC strategy for chunk scheduling. In total, we obtain 21 candidate implementations with double precision for all SpMV formats and parameters.

2) *GPU*: On the GPU side, our framework incorporates a total of seven sparse formats, establishing a comprehensive evaluation baseline. This set includes: 1) custom high-performance kernels developed for the ELL, SELL-*c*- σ , and SELL-*c*-R formats; 2) the default implementations for COO,

CSR, and Sliced ELL (S-ELL) from the cuSPARSE library (v12.3.0); and 3) a publicly available CSR5 implementation¹.

This setting is intentional, striking a balance between two critical performance trade-offs: structured regularity, as seen in classic ELL, and adaptive compactness, as in ELL's sliced/ordered derivatives. By including both hand-optimized kernels and established vendor libraries, we ensure a rigorous and fair evaluation of our intelligent selector across a diverse and challenging design space. Our custom kernels incorporate several optimizations, including warp-level load balancing, explicit vectorization using *float4* operations, and shared-memory caching. All GPU experiments are conducted in single precision, with kernel launch configurations fine-tuned per matrix to achieve maximum occupancy.

D. Model and Training

Based on practical experiments, the first layer of our multimodal convolution utilizes 16 filters of size 3 for local feature extraction, and the second layer has 16 stride-2 filters of size 5 for global feature aggregation. The FFNN employs a two-layer feedforward network to embed expert-designed features into 32 dimensions. After concatenation, the MLP with a structure of (512, 256, 32) summarizes the multimodal features of the sparse matrix. A *softmax* function is then applied to compute the probabilities for all candidate SpMV implementations. The model is trained using *categorical cross-entropy* as the loss function. Given the different sizes of the candidate format sets and the varying sample counts in the CPU and GPU datasets, we adjust the training duration accordingly. Specifically, the GPU model is trained for 100 epochs, while the CPU model undergoes 512 epochs. Other hyperparameters remain consistent across both models: *lr* = 0.001 and *batches* = 64.

E. Results of MM-Adapter

1) *Training Loss and Accuracy*: Figure 14 displays the accuracy and loss curves of training and validation on CPU side, with transparency indicating variability in multiple experiments. As training progresses, the loss of MM-Adapter

¹https://github.com/Rhythmicc/CSR5_CUDA_Lib

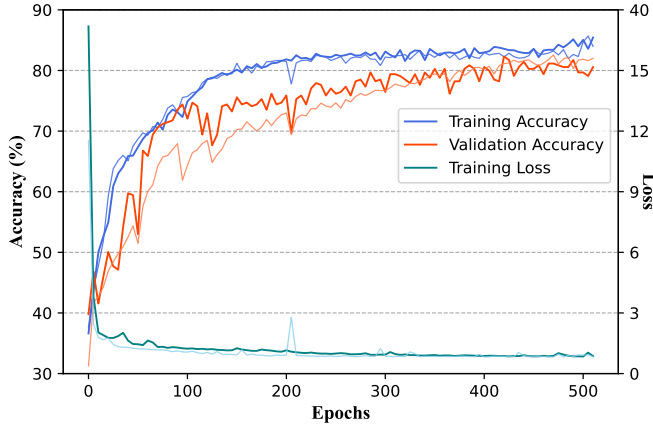


Fig. 14. Loss and accuracy curve of the MM-Adapter on CPU.

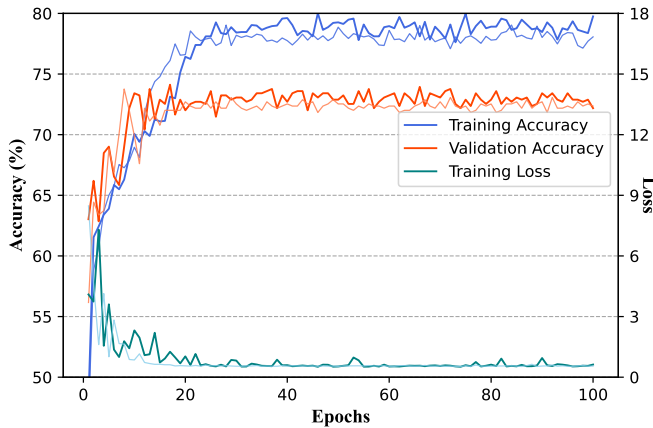


Fig. 15. Loss and accuracy curve of the MM-Adapter on GPU.

decreases steadily. After 500 epochs, both the training and validation datasets achieve accuracies exceeding 80%. Extensive tests illustrate that for this multi-classification task, the final loss stabilizes at approximately 0.3, and the average training and validation accuracies reach 85.08% and 82.06%, respectively. For GPU, Figure 15 shows convergence after around 50 epochs at loss 0.55, yielding 78.06% training accuracy and 73.24% validation accuracy. These outcomes verify MM-Adapter rapidly learns multimodal matrix features and converges efficiently.

TABLE II
COMPARISON OF PREDICTION RESULTS WITH OTHER FRAMEWORKS ON THE TEST SET.

Framework	Precision(%)		Recall(%)		Accuracy(%)	
	CPU	GPU	CPU	GPU	CPU	GPU
SMAT [20]	15.92	54.62	13.24	28.39	30.69	64.08
DIESEL [9]	46.72	66.12	48.30	66.20	62.77	66.20
SpNet [24], [30]	41.21	59.71	32.92	60.74	75.88	60.74
MatNet [36]	52.86	63.34	46.76	67.08	72.90	67.08
MM-Adapter	62.63	73.61	59.41	73.87	81.05	74.12

2) *Overall Prediction Study*: We now evaluate the overall predictive performance of the MM-Adapter on the test set and compare it with various established frameworks, with results detailed in Table II. Beyond *accuracy*, the table also

TABLE III
ABLATION STUDY OF MM-ADAPTER ON CPU PLATFORM.

Variant	Precision(%)	Recall(%)	Accuracy(%)
only channel C1	56.67	51.05	75.94
w/o MLP	60.55	50.47	79.90
w/o Modality 1	56.85	54.72	75.35
w/o Modality 1&3	56.18	51.08	72.63
w/o Modality 2	36.48	31.54	70.15
w/o Expert Features	57.19	51.47	73.51

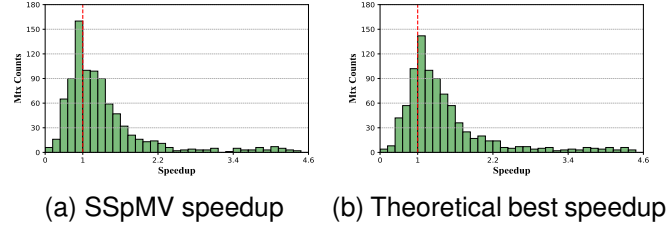


Fig. 16. Distribution of speedup over the Intel MKL Inspector-Executor.

documents the *precision* and *recall* of all frameworks, which are essential performance metrics for classification models. For CPU implementation selection, MM-Adapter outperforms competing methods with 62.63% precision, 59.41% recall and 81.05% accuracy. Conventional SMAT (Decision Tree) and DIESEL (MLP) struggle over 21 SpMV candidates, while CNN-based SpNet and MatNet suffer poor generalization for heterogeneous matrix features. On GPU, MM-Adapter attains 73.61% precision and 74.12% accuracy. Consistent superiority across CPU and GPU confirms the multimodal design's efficacy for cross-hardware SpMV selection.

Moreover, we conduct an ablation study to dissect the architecture of MM-Adapter, with results presented in Table III. From the results, we can assess the effectiveness of each component: 1) First, the multiple channels enhance the MM-Adapter's capabilities, facilitating the extraction of implicit structural information from raw data. 2) Second, although MLP does not markedly improve accuracy, it helps the model to be more comprehensive when searching for positive examples (recall). 3) Finally, among the three modalities, **M2: distribution** is identified as the most crucial, as it harbors the richest structural information, pivotal for the model's performance. Modalities 1 and 3 together seem to make a similar contribution to the model as the expert-designed feature sets. In conclusion, the ablation study proves the rationality of the multimodal approaches and the effectiveness of our MM-Adapter.

F. Performance Analysis

1) *Platform-wise Speedup Distribution*: Figure 16(a) shows the speedup distribution of our SSpMV on CPUs, with the baseline performance measured against the *mkl_sparse_d_mv*. Here, the SpMV kernel is called using the format and parameters predicted by MM-Adapter. Owing to the high prediction accuracy, SSpMV consistently delivers significant speedups across most tested sparse matrices. Specifically, the speedup ratio ranges from a minimum of $0.08\times$ to a maximum of $46.8\times$, with a geometric speedup of $1.26\times$. This result on the Intel Xeon platform, combined with the $2.07\times$ geometric

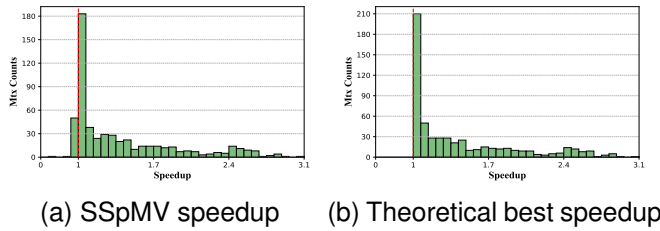


Fig. 17. Distribution of speedup over the NVIDIA cuSPARSE.

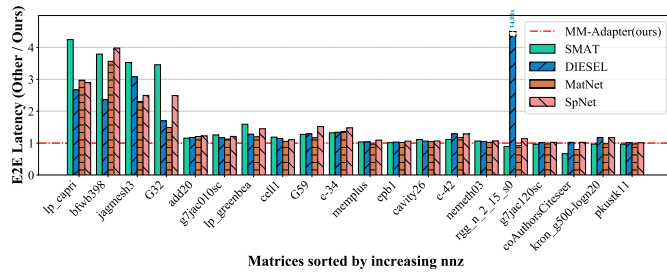


Fig. 18. End-to-End Execution Time Comparison with Four Baseline Frameworks.

speedup over ArmPL on the Huawei Kunpeng 920 (detailed in Table IV), establishes a consistent performance baseline across diverse CPU architectures. To demonstrate the availability of SSpmV, Figure 16(b) shows the results for hand-pick fastest SpMV implementation. Here, the worst and best results are $0.15\times$ and $46.8\times$, with a geometric speedup of $1.45\times$. We can see that the two figures exhibit overall similar distributions, with the primary difference concentrated in the $[0.4, 1.24]$ interval. The primary cause of performance deterioration is incorrect format and hyperparameter prediction, leading to inefficient computation patterns.

Similar GPU trends appear in Figure 17(a): speedup ratios range from $0.21\times$ to $3.33\times$, with a geometric mean of $1.31\times$. Although a few matrices show degradation, most achieve speedups exceeding $1.0\times$. Notably, since cuSPARSE CSR is the baseline, the theoretical distribution in Figure 17(b) has no matrices left of $x = 1$, averaging $1.35\times$ speedup.

2) *Robustness of MM-Adapter Selection:* Quantitative statistics on the histogram distributions in Figure 16 explicitly verify the model’s robustness: for the predicted results in Figure 16(a), the numbers of matrices falling into five successive bins within the $[0,1]$ speedup range are 6, 16, 65, 90 and 160, while the corresponding values from Figure 16(b) are 4, 8, 42, 57 and 102; the close variation tendency of matrix distribution between predicted and theoretical cases reveals that the performance loss induced by occasional wrong prediction is close to the inherent performance gap of native SpMV algorithms. As illustrated in Figure 17(a), the matrix counts distributed over the same five subranges below $1.0\times$ are 0, 1, 0, 1, 50, implying less severe performance degradation from misprediction on GPU compared with CPU platforms.

Additionally, we statistically collect the GFLOPs of the top-1 and top-2 candidate SpMV implementations predicted by MM-Adapter on the Intel Xeon CPU platform, and calculate the GFLOPs ratio between the suboptimal and optimal

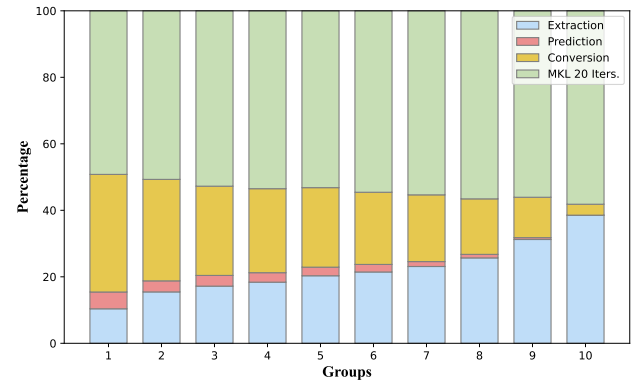


Fig. 19. Overhead of several matrix groups on CPU platform.

selections. The averaged ratio reaches $0.79\times$ across all test matrices, which demonstrates that MM-Adapter can effectively map diverse sparse structural features to appropriate high-performance SpMV implementations.

3) *End-to-End Latency Evaluation:* To further validate practical efficiency, we conduct an end-to-end latency analysis on 20 random sparse matrices (nnz from 2K to 80M). The measured end-to-end runtime covers feature extraction, model inference and 100 repeated SpMV executions, while format conversion overhead is excluded, as such conversion costs depend on storage formats rather than the intrinsic design of prediction models. As shown in Figure 18 (sorted by ascending nnz), MM-Adapter outperforms four baselines at small to medium scales. For the first six matrices (nnz < 20K), geometric mean relative latencies are 2.56 (SMAT), 1.88 (DIESEL), 1.92 (MatNet), and 2.18 (SpNet). For the middle eight matrices (30K–300K), the values drop to 1.19, 1.18, 1.12, and 1.24, respectively.

For the largest six matrices ($\text{nnz} > 400\text{K}$), the geometric means become 0.91 (SMAT), 1.05 (DIESEL), 0.94 (MatNet), 1.07 (SpNet), with minima 0.66, 1.02, 0.80, 1.01 respectively (DIESEL’s mean excludes an outlier 14.04 due to an extremely inefficient kernel). At this scale, MatNet and SMAT occasionally outperform ours because their lighter predictors result in lower feature extraction overhead, which is a major component of end-to-end latency (see Section V-G). Across all test samples in the entire CPU dataset, the averaged feature extraction cost of MatNet and SMAT is respectively $0.87\times$ and $0.82\times$ that of MM-Adapter, quantitatively verifying their advantage in preprocessing expenditure. Overall, MM-Adapter remains highly competitive for sparse matrices with small to medium-sized nnz .

G. Preprocessing Overhead

To provide a comprehensive overhead analysis, we examine the time breakdown on both CPU and GPU using the same set of randomly selected sparse matrices. The preprocessing overhead of SSpMV consists of three components: extracting expert-designed features and generating multimodal representations, predicting the optimal SpMV implementation via MM-Adapter, and converting from CSR to alternative formats.

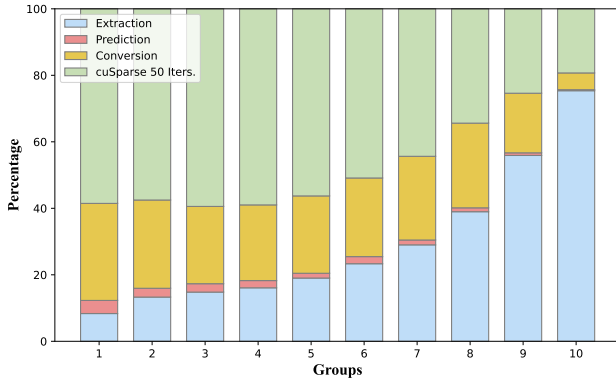


Fig. 20. Overhead of several matrix groups on GPU platform.

TABLE IV
SSpMV RESULTS ON DIFFERENT ARCHITECTURES

Platform	Method	Accuracy (%)	Speedup
Intel Xeon 5120	CSR	90.25	-
	S-ELL	87.27	1.29×
	DIA	83.10	6.01×
	CSR5	78.39	1.22×
	Others	67.93	1.11×
Nvidia A100-40GB	CSR	73.78	-
	S-ELL	80.28	1.31×
	ELL	80.33	1.98×
	Others	55.87	1.09×
Huawei Kunpeng 920	CSR	92.27	-
	S-ELL	89.65	2.22×
	DIA	85.31	1.32×
	CSR5	76.55	4.70×
	Others	65.37	1.35×

Figure 19 and Figure 20 present the overhead breakdowns across ten matrix groups with increasing average nnz . The first and third components scale with nnz , while the prediction step requires approximately 3.19 ms per matrix. On both platforms, the relative burden of format conversion diminishes as workload increases, while feature extraction constitutes a larger share of the preprocessing overhead. For matrices with nnz exceeding one million, the average preprocessing overhead of SSpMV is equivalent to 7.84 MKL iterations. Although this absolute overhead is identical across hardware, its relative proportion differs significantly between platforms. Since cuSparse demonstrates superior scaling efficiency on large-scale matrices compared to MKL, with average kernel execution times of MKL being 8.3 $\times$, 12.1 $\times$, and 21.3 $\times$ that of cuSparse in the last three matrix groups, the faster kernel execution on GPU results in a higher relative share of preprocessing overhead. Conversely, the longer MKL execution time on CPU reduces the proportional contribution of preprocessing. As the preprocessing steps are hardware-independent and iterative solvers typically require hundreds to thousands of SpMV operations, this initial cost is effectively amortized over successive iterations, ensuring that SSpMV delivers consistent performance gains on both CPU and GPU platforms. In practical deployment within such iterative solvers, caching the inference results is essential to amortize the preprocessing overhead across iterations.

H. Portability Discussion

Building upon experiments across Intel Xeon, Huawei Kunpeng 920 (ARM), and NVIDIA A100, we further summarize the portability of SSpMV across these hardware architectures. Table IV presents the accuracy and geometric speedup compared to the CSR baseline, focusing solely on format type evaluation. The results demonstrate that, while format preferences vary across architectures, CSR and S-ELL series remain the most commonly selected formats on all evaluated platforms. Although category imbalance in the training data may cause slight fluctuations in prediction accuracy when deploying the same dataset on different hardware, the consistently high accuracy and notable speedup across all three platforms affirm that SSpMV is an effective and portable high-performance framework for sparse matrix computations.

VI. RELATED WORK

Sparse kernels often struggle to maintain high-performance consistency in various applications due to input sensitivity [5], [41], [42]. For SpMV, numerous approaches have been proposed to optimize vectorization and load balancing. Much previous work tends to design new formats coupled with hardware architectures to optimize computational efficiency, such as SELL-c- σ [12], [43], CSR5 [13], LAV [15], etc. In order to broaden the adaptability of singular formats, some hybrid formats have also been created, such as OSKI [44], Cocktail [45], and HYB [46]. These hybrids typically rely on empirical statistics or interpolation methods to estimate optimal parameter configurations, which may constrain their scalability.

Therefore, some recent efforts leverage ML models to make more accurate predictions of SpMV performance. For instance, the DIESEL [9] utilizes a six-layer neural network to predict the best SpMV kernel, while SMAT [20] builds a C5.0 decision tree to select the optimal format. Note that these models both use nnz features related to the particular format for training and prediction. In contrast, WISE [21] tries to bridge the gap between SpMV parallelization and nnz distribution. It innovatively proposes skew and locality features and constructs a speedup decision tree for performance modeling. However, WISE makes the overhead evaluation inaccurate by ignoring the inference time of 29 decision tree models. Besides focusing on pure software kernel selection, Yadav et al. [47] adopt lightweight decision trees to dynamically select optimal SpGEMM dataflow for FPGA accelerators and introduce an extra reconfiguration engine to balance switching overhead and performance gain. Unlike previous studies, Zhao et al. [25] first used CNN to extract latent sparse structures and achieved considerable precision. Following this, Xie [36] proposed MatNet to predict the performance of SpGEMM based on the density representation. Pichel [24], [30] also designed SpNet with RGB channels to further enhance feature representations.

To our knowledge, SSpMV is the first to apply multi-modal machine learning to sparse HPC challenges. Inspired by WISE's feature sampling, its three-modality representations better characterize SpMV parallel properties. Moreover, we

consider the broadest array of non-hybrid formats and parameters. Experimental results demonstrate that sparse kernels benefit from multimodal machine learning.

VII. CONCLUSION

In this paper, we propose a set of high-performance SpMV algorithms for various widely used formats and design a sparsity-aware auto-tuning framework called SSPMV that can adaptively determine the optimal format and parameters for a given sparse matrix. By integrating expert-designed features with innovative multimodal representations, our MM-Adapter achieves state-of-the-art prediction accuracy. Empirical results demonstrate that SSPMV achieves a geometric speedup of $1.26\times$ on Intel CPUs and $1.31\times$ on GPUs, closely approaching the theoretical maximum of $1.45\times$ and $1.35\times$, respectively. Furthermore, the portability analysis confirms that SSPMV can achieve substantial performance gains across different hardware platforms, validating its effectiveness and versatility as a cross-architecture optimization framework.

REFERENCES

- [1] M. Tilenius, E. Larsson, E. Lehto, and N. Flyer, "A task parallel implementation of a scattered node stencil-based solver for the shallow water equations," in *Proc. 6th Swedish Workshop on Multi-Core Computing, Halmstad University, Halmstad, Sweden*, 2013, pp. 33–36.
- [2] C. Chen, K. Li, A. Ouyang, Z. Zeng, and K. Li, "Gflink: An in-memory computing architecture on heterogeneous cpu-gpu clusters for big data," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 6, pp. 1275–1288, 2018.
- [3] L. Wang, J. Zhan, C. Luo, Y. Zhu, Q. Yang, Y. He, W. Gao, Z. Jia, Y. Shi, S. Zhang *et al.*, "Bigdatabench: A big data benchmark suite from internet services," in *2014 IEEE 20th international symposium on high performance computer architecture (HPCA)*. IEEE, 2014, pp. 488–499.
- [4] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai *et al.*, "Deep graph library: A graph-centric, highly-performant package for graph neural networks," *arXiv preprint arXiv:1909.01315*, 2019.
- [5] K. Asanovic, R. Bodik, B. C. Catanzaro, J. J. Gebis, P. Husbands, K. Keutzer, D. A. Patterson, W. L. Plishker, J. Shalf, S. W. Williams *et al.*, "The landscape of parallel computing research: A view from berkeley," 2006.
- [6] Y. Saad, *Iterative methods for sparse linear systems*. SIAM, 2003.
- [7] S. Brin and L. Page, "The anatomy of a large-scale hypertextual web search engine," *Computer networks and ISDN systems*, vol. 30, no. 1-7, pp. 107–117, 1998.
- [8] G. L. Sleijpen and H. A. Van der Vorst, "A jacobi-davidson iteration method for linear eigenvalue problems," *SIAM review*, vol. 42, no. 2, pp. 267–293, 2000.
- [9] T. Mohammed, A. Albeshri, I. Katib, and R. Mehmood, "Diesel: A novel deep learning-based tool for spmv computations and solving sparse linear equation systems," *The Journal of Supercomputing*, vol. 77, no. 6, pp. 6313–6355, 2021.
- [10] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proceedings of the conference on high performance computing networking, storage and analysis*, 2009, pp. 1–11.
- [11] X. Liu, M. Smelyanskiy, E. Chow, and P. Dubey, "Efficient sparse matrix-vector multiplication on x86-based many-core processors," in *Proceedings of the 27th international ACM conference on International conference on supercomputing*, 2013, pp. 273–282.
- [12] M. Kreutzer, G. Hager, G. Wellein, H. Fehske, and A. R. Bishop, "A unified sparse matrix data format for efficient general sparse matrix-vector multiplication on modern processors with wide simd units," *SIAM Journal on Scientific Computing*, vol. 36, no. 5, pp. C401–C423, 2014.
- [13] W. Liu and B. Vinter, "Csr5: An efficient storage format for cross-platform sparse matrix-vector multiplication," in *Proceedings of the 29th ACM on International Conference on Supercomputing*, 2015, pp. 339–350.
- [14] B. Xie, J. Zhan, X. Liu, W. Gao, Z. Jia, X. He, and L. Zhang, "Cvr: Efficient vectorization of spmv on x86 processors," in *Proceedings of the 2018 International Symposium on Code Generation and Optimization*, 2018, pp. 149–162.
- [15] S. Yesil, A. Heidarshenas, A. Morrison, and J. Torrellas, "Speeding up spmv for power-law graph analytics by enhancing locality & vectorization," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–15.
- [16] A. Buttari, V. Eijkhout, J. Langou, and S. Filippone, "Performance optimization and modeling of blocked sparse kernels," *The International Journal of High Performance Computing Applications*, vol. 21, no. 4, pp. 467–484, 2007.
- [17] N. Bell and M. Garland, "Efficient sparse matrix-vector multiplication on cuda," Nvidia Technical Report NVR-2008-004, Nvidia Corporation, Tech. Rep., 2008.
- [18] D. Didona, F. Quaglia, P. Romano, and E. Torre, "Enhancing performance prediction robustness by combining analytical modeling and machine learning," in *Proceedings of the 6th ACM/SPEC international conference on performance engineering*, 2015, pp. 145–156.
- [19] P. Mpakos, D. Galanopoulos, P. Anastasiadis, N. Papadopolou, N. Koziris, and G. Goumas, "Feature-based spmv performance analysis on contemporary devices," in *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2023, pp. 668–679.
- [20] J. Li, G. Tan, M. Chen, and N. Sun, "Smat: An input adaptive auto-tuner for sparse matrix-vector multiplication," in *Proceedings of the 34th ACM SIGPLAN conference on Programming language design and implementation*, 2013, pp. 117–126.
- [21] S. Yesil, A. Heidarshenas, A. Morrison, and J. Torrellas, "Wise: Predicting the performance of sparse matrix vector multiplication with machine learning," in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2023, pp. 329–341.
- [22] S. Chen, J. Fang, C. Xu, and Z. Wang, "Adaptive hybrid storage format for sparse matrix-vector multiplication on multi-core simd cpus," *Applied Sciences*, vol. 12, no. 19, p. 9812, 2022.
- [23] A. Benatia, W. Ji, Y. Wang, and F. Shi, "Machine learning approach for the predicting performance of spmv on gpu," in *2016 IEEE 22nd International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2016, pp. 894–901.
- [24] J. C. Pichel and B. Pateiro-López, "A new approach for sparse matrix classification based on deep learning techniques," in *2018 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2018, pp. 46–54.
- [25] Y. Zhao, J. Li, C. Liao, and X. Shen, "Bridging the gap between deep learning and sparse matrix format selection," in *Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming*, 2018, pp. 94–108.
- [26] T. A. Davis and Y. Hu, "The university of florida sparse matrix collection," *ACM Transactions on Mathematical Software (TOMS)*, vol. 38, no. 1, pp. 1–25, 2011.
- [27] F. Hecht, "New development in freefem++," *Journal of numerical mathematics*, vol. 20, no. 3-4, pp. 251–266, 2012.
- [28] H. Jasak, A. Jemcov, Z. Tukovic *et al.*, "Openfoam: A c++ library for complex physics simulations," in *International workshop on coupled methods in numerical dynamics*, vol. 1000, 2007, pp. 1–20.
- [29] W. Kim, H. Kim, J. Lee, H. Kim, and J.-H. Kim, "Multi-mode spmv accelerator for transprecision pagerank with real-world graphs," *IEEE Access*, vol. 11, pp. 6261–6272, 2023.
- [30] J. C. Pichel and B. Pateiro-Lopez, "Sparse matrix classification on imbalanced datasets using convolutional neural networks," *IEEE Access*, vol. 7, pp. 82377–82389, 2019.
- [31] A. Monakov, A. Lokhtov, and A. Avetisyan, "Automatically tuning sparse matrix-vector multiplication for gpu architectures," in *High Performance Embedded Architectures and Compilers: 5th International Conference, HiPEAC 2010, Pisa, Italy, January 25-27, 2010. Proceedings 5*. Springer, 2010, pp. 111–125.
- [32] D. Kirk, "Nvidia cuda software and gpu parallel computing architecture," in *Proceedings of the 6th International Symposium on Memory Management*, 2007, p. 103–104.
- [33] X. Yang, S. Parthasarathy, and P. Sadayappan, "Fast sparse matrix-vector multiplication on gpus: Implications for graph mining," *arXiv preprint arXiv:1103.2405*, 2011.
- [34] Y. Funk, M. Götz, and H. Anzt, "Prediction of optimal solvers for sparse linear systems using deep learning," in *Proceedings of the 2022 SIAM Conference on Parallel Processing for Scientific Computing*. SIAM, 2022, pp. 14–24.

- [35] R. Dorfman, "A formula for the gini coefficient," *The review of economics and statistics*, pp. 146–149, 1979.
- [36] Z. Xie, G. Tan, W. Liu, and N. Sun, "Ia-spgemm: An input-aware auto-tuning framework for parallel sparse matrix-matrix multiplication," in *Proceedings of the ACM International Conference on Supercomputing*, 2019, pp. 94–105.
- [37] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, "Multimodal machine learning: A survey and taxonomy," *IEEE transactions on pattern analysis and machine intelligence*, vol. 41, no. 2, pp. 423–443, 2018.
- [38] W. Guo, J. Wang, and S. Wang, "Deep multimodal representation learning: A survey," *IEEE Access*, vol. 7, pp. 63 373–63 394, 2019.
- [39] W. Wan, L. Gan, W. Wang, Z. Yin, H. Tian, Z. Zhang, Y. Wang, M. Hua, X. Liu, S. Xiang *et al.*, "7-pflops extreme scale earthquake simulation with crossing multi-faults and topography on sunway," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–15.
- [40] R. T. Mills, M. Adams, S. Balay, J. Brown, J. Faibussowitsch, T. Isaac, M. Knepley, T. Munson, H. Suh, S. Zampini *et al.*, "Petsc/tao developments for early exascale systems," *arXiv preprint arXiv:2406.08646*, 2024.
- [41] A. Venkat, M. S. Mohammadi, J. Park, H. Rong, R. Barik, M. M. Strout, and M. Hall, "Automating wavefront parallelization for sparse matrix computations," in *SC'16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2016, pp. 480–491.
- [42] I. Nisa, C. Siegel, A. S. Rajam, A. Vishnu, and P. Sadayappan, "Effective machine learning based format selection and performance modeling for spmv on gpus," in *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2018, pp. 1056–1065.
- [43] C. Gómez, F. Mantovani, E. Focht, and M. Casas, "Efficiently running spmv on long vector architectures," in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 292–303.
- [44] R. Vuduc, J. W. Demmel, and K. A. Yelick, "Oski: A library of automatically tuned sparse matrix kernels," in *Journal of Physics: Conference Series*, vol. 16, no. 1. IOP Publishing, 2005, p. 521.
- [45] B.-Y. Su and K. Keutzer, "clspmv: A cross-platform opencl spmv framework on gpus," in *Proceedings of the 26th ACM international conference on Supercomputing*, 2012, pp. 353–364.
- [46] W. Yang, K. Li, and K. Li, "A hybrid computing method of spmv on cpu-gpu heterogeneous computing systems," *Journal of Parallel and Distributed Computing*, vol. 104, pp. 49–60, 2017.
- [47] S. Yadav, A. Namjoo, and B. Asgari, "Misam: Machine learning assisted dataflow selection in accelerators for sparse matrix multiplication," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, p. 824–838.

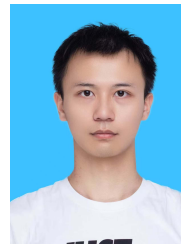


Guoqing Xiao (Member, IEEE) received his Ph.D. degree in Computer Science and Technology from the College of Computer Science and Electronic Engineering (CSEE) at Hunan University (HNU) in China in 2017. He is currently a Professor at the HNU. He worked as a Postdoctoral Research Fellow at the Data Systems Group of the David R. Cheriton School of Computer Science, University of Waterloo, Canada, during 2017 to 2019. His main research interests are on high-performance computing and AI computing. He has published more than 50 papers

in peer-reviewed international journals and conferences, such as ISCA, DAC, ICDE, IEEE TC/TPDS/TKDE/TII, ACM TOPC/TORS/CSUR, etc.



Shijie Zhao received the B.S. degree from the School of Computer Science and Technology, China University of Mining and Technology, Xuzhou, China, in 2024. Now he is a Master's student at Hunan University. His research interests include parallel computing and artificial intelligence.



Qin Du received the bachelor's degree from Hunan University of Science and Technology, China, in 2023. Now he is a Ph.D with HNU. His main research interests are on high-performance computing and AI computing. His research interests include high-performance computing, parallel and distributed computing.



Shengle Lin received the Ph.D. degree in Computer Science and Technology from the College of Information Science and Electronic Engineering, Hunan University, Changsha, China, in 2024. From 2023 to 2024, he was a joint Ph.D. student at the Agency for Science, Technology and Research (A*STAR), Singapore. He is currently a postdoctoral fellow at Hunan University. His research interests include high-performance computing, parallel computing, numerical computation, and artificial intelligence.



Yuedan Chen (Member, IEEE) is currently an Associate Professor with the Central South University (CSU), China. She is also a Visiting Ph.D. Student and a Postdoctoral Research Fellow with the Data Systems Group, David R. Cheriton School of Computer Science, University of Waterloo, Canada. She has published more than 30 papers in peer-reviewed international journals and conferences, such as IEEE ICDE/IEEE Transactions on Parallel and Distributed Systems/IEEE Internet of Things Journal, ACM TOPC/CSUR, etc. Her research interests include

high-performance computing, parallel, and distributed processing. She is a member of ACM and CCF.



Wangdong Yang received the PhD degree in Computer Science from Hunan University, China, and the MS degree in Computer Science from Central South University, China. He is a professor of computer science and technology at Hunan University, China. His research interests include modeling and programming for heterogeneous computing systems, parallel and distributed computing, and numerical computation. He has published more than 60 papers in international conferences and journals. He is currently served on the editorial boards of the *IEEE Internet of Things Journal*.

Internet of Things Journal.



Kenli Li (Senior Member, IEEE) received the PhD degree in computer science from the Huazhong University of Science and Technology, China (2003). He was a visiting scholar with the University of Illinois with Urbana-Champaign (2004–2005). He is currently a Cheung Kong professor of computer science and technology with Hunan University (HNU), the Vice-President of the HNU, the Dean of the College of Computer Science and Electronic Engineering of HNU, and the Director in the National Supercomputing Center in Changsha. His major research interests

include parallel and distributed processing, high-performance computing, and Big Data management. He has published over 350 research papers in international conferences/journals. He is a Fellow of the CCF. He is currently serving or has served as an associate editor for the *IEEE Transactions on Computers*, the *IEEE Transactions on Industrial Informatics*, and the *IEEE Transactions on Sustainable Computing*.